Database Course

Ahmad Yoosofan

University of Kashan

Data or Quantitative data

Quantitative data are

- measures of values or counts and are expressed as numbers.
- data about numeric variables (e.g. how many; how much; or how often).
- measures of 'types' and may be represented by a name, symbol, or a number code.
- data about categorical variables (e.g. what type) abs.gov.au .

Quantitative Data Examples

- I updated my phone *6 times* in a quarter.
- My teenager grew by *3 inches* last year.
- *83 people* downloaded the latest mobile application.
- My aunt lost *18 pounds* last year.
- *150 respondents* were of the opinion that the new product feature will not be successful.
- There will be *30% increase* in revenue with the inclusion of a new product.
- *500 people* attended the seminar.
- *54% people* prefer shopping online instead of going to the mall.
- She has *10 holidays* in this year.
- Product X costs *$1000*.

Advantages of Quantitative Data

- Conduct in-depth research: Since quantitative data can be statistically analyzed, it is highly likely that the research will be detailed.
- Minimum bias: There are instances in research, where personal bias is involved which leads to incorrect results. Due to the numerical nature of quantitative data, the personal bias is reduced to a great extent.
- Accurate results: As the results obtained are objective in nature, they are extremely accurate.

Models and their Usage

- What is a model?
- Why do we need model?
- A little history of model in mathetamtics
- Real life situations and models

Old models for Storing Data

- Why do we need secondary storage for preserving data?
- Direct Programming with Files (by Old languages like PL/I and Cobol)
- Hierarchical model
- Network model

„ :

https://mariadb.com/kb/en/database-theory/

Relational Model

- Table
- Field
- Record

Sales DataBase (1)

product_id	product_name	stock	quantity	customer_id	customer_name
10	Sugar	20	2	32	Ali
11	Salt	100	5	32	Ali
11	Salt	100	6	33	Reza
10	Sugar	20	3	34	Karim

Sales DataBase (2)

Product Table

product_id	product_name	stock
10	Sugar	20
11	Salt	100

Sales DataBase (3)

Customer Table

customer_id	customer_name
32	Ali
33	Reza
34	Karim

Sales DataBase (4)

Sales Table

product_id	customer_id	quantity
10	32	2
11	32	5
11	33	6
10	34	3

Sales DataBase (5)

- Product Table: (product_id, product_name, stock)
- Customer Table: (customer_id, customer_name)
- Sales Table: (product_id, customer_id , quantity)

Parts Suppliers Database

- P(pn, pname, color, weight, city)
- S(sn, sname, status, city)
- SP(sn, pn, qty)

Parts, Suppliers and Projects Database

- S(sn ,sname,status,city)
- P(pn ,pname,color,weight,city)
- J(jn ,jname,budget,city)
- SPJ(sn , pn , jn ,qty)

جدول فروش یک فروشگاه

میزان فروش	شماره فاکتور فروش	موجودی	نام جنس	شماره جنس
12	1	1000	شکر	1
14	4	1000	شکر	1
5	5	1000	شکر	1
10	2	1500	نمک	2
20	4	1500	نمک	2
30	5	1500	نمک	2
40	3	1500	نمک	2

فروشگاه ۲

میزان فروش	شماره فاکتور فروش	شماره جنس
12	1	1
14	4	1
5	5	1
10	2	2
20	4	2
30	5	2
40	3	2

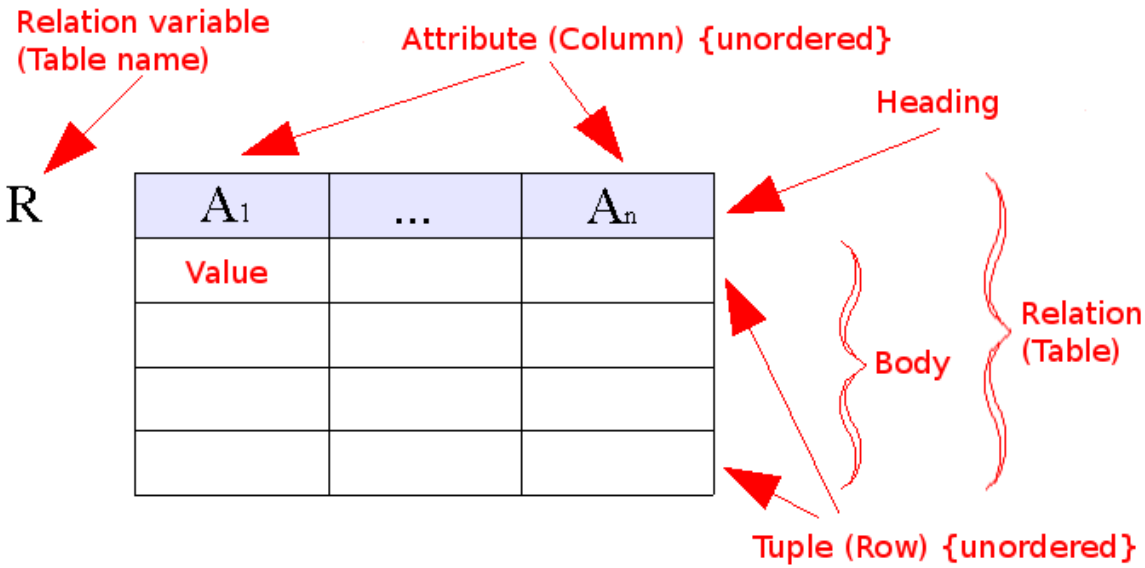
موجودی	نام جنس	شماره جنس
1000	شکر	1
1500	نمک	2

Types

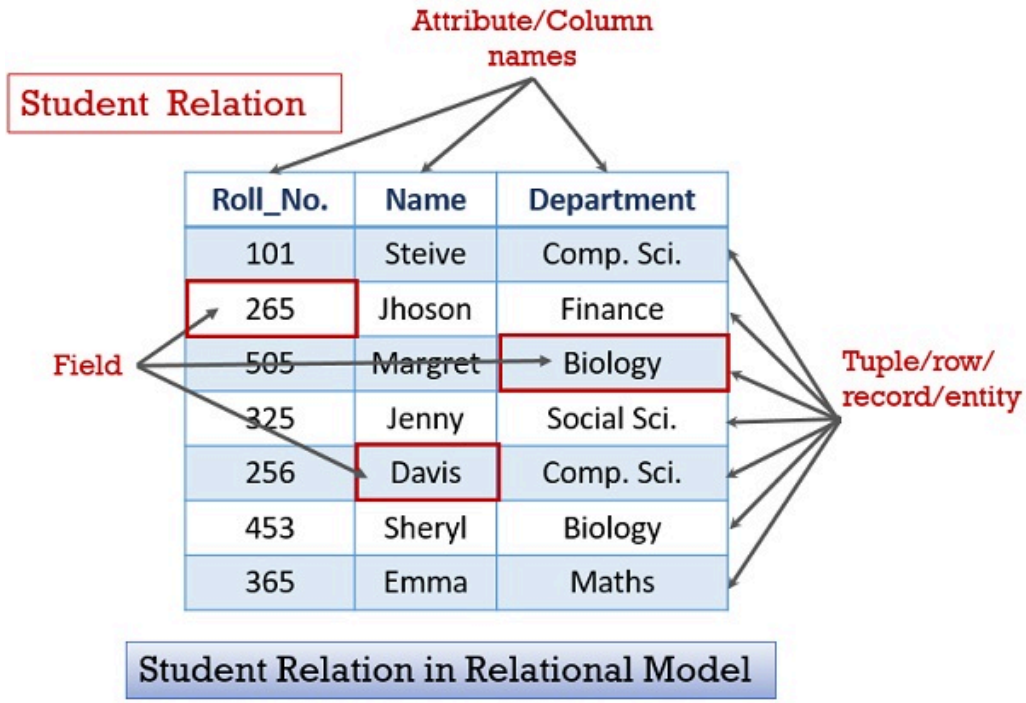
Not exact types

- product_id: positive number
- product_name: string
- stock: positive number
- quantity: positive number
- customer_id: positive number
- customer_name: string

Relational Model Like Table



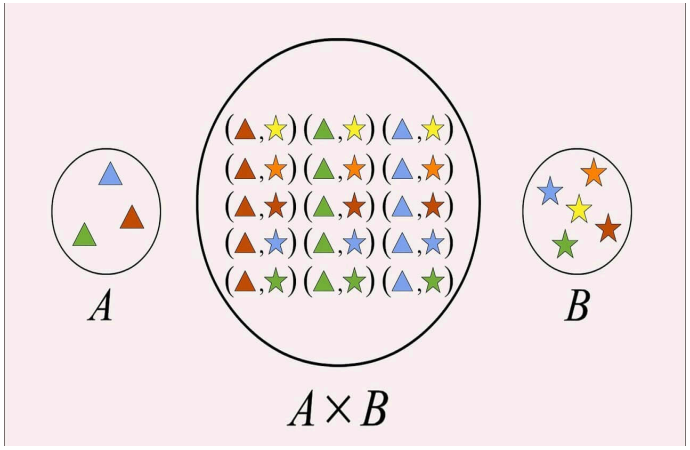
https://en.wikipedia.org/wiki/Relational_model



https://binaryterms.com/relational-data-model.html

Cartesian Product(I)

$$X \times Y = \{ (x, y) \mid x \in X \wedge y \in Y \}$$

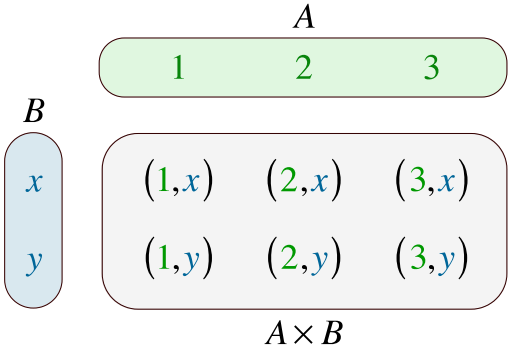


https://smartinsight.ir/ضرب-دکارتی-دو-مجموعه/

Cartesian Product(I)

$A \times B = \{ (a, b) \mid a \in A \wedge b \in B\}$

A={1, 2, 3} and B={x, y}



$A \times B = \{(1, x), (1, y), (2, x), (2, y), (3, x), (3, y)\}$

$B \times A = \{(x, 1), (x, 2), (x, 3), (y, 1), (y, 2), (y, 3)$

$A \times B \neq B \times A$

https://math24.net/cartesian-product-sets.html

Set of Tuples (1)

```
{
  ( a11, a12, a13, ... , a1n ) , # Tuple 1
  ( a21, a22, a23, ... , a2n ) , # Tuple 2
          ....
  ( am1, am2, am3, ... , amn )   # Tuple m
}
```

- Domain of ({a11, a21, ... , am1}) = S1
- Domain of ({a12, a22, ... , am2}) = S2
- ...
- Domain of ({a1n, a2n, ... , amn}) = Sn

R1 ⊆ S1 * S2 * ... * Sn

- {a11, a21, ... , am1} ⊆ S1
- {a12, a22, ... , am2} ⊆ S2
- ... ⊆ Si
- {a1n, a2n, ... , amn} ⊆ Sn

Set of Tuples (2)

```
{
  { a11 : Label_1 , a12 : Label_2 , a13 : Label_3 , ... , a1n : Label_n } , # Tuple 1
  { a21 : Label_1 , a22 : Label_2 , a23 : Label_3 , ... , a2n : Label_n } , # Tuple 2
          ....
  { am1 : Label_1 , am2 : Label_2 , am3 : Label_3 , ... , amn : Label_n } , # Tuple m
}
```

Set of Tuples (3)

```
{
  { a21 : Label_1 , a22 : Label_2 , a23 : Label_3 , ... , a2n : Label_n } , # Tuple 2
  { a11 : Label_1 , a12 : Label_2 , a13 : Label_3 , ... , a1n : Label_n } , # Tuple 1
          ....
  { am1n : Label_1 , am2n : Label_2 , am3n : Label_3 , ... , amn : Label_n } , # Tuple m
}
```

Set of Tuples (4)

```
{
  { a21 : Label_1 , a22 : Label_2 , a23 : Label_3 , ... , a2n : Label_n } , # Tuple 2
  { a12 : Label_2 , a13 : Label_3 , a11 : Label_1 ,  ... , a1n : Label_n } , # Tuple 1
          ....
  { am1 : Label_1 , am2 : Label_2 , am3 : Label_3 , ... , amn : Label_n } , # Tuple m
}
```

Relational Algebra

http://yoosofan.github.io/webrel/index.html

http://yoosofan.github.io/webrel/help/help.html

- Schema
- Table Schema
- Database Schema

```
p;
------

s minus s ;

------

RELATION {
  TUPLE{
    PN PN("s4"), SNAME SNAME("Clark"),
    STATUS STATUS(20), CITY CITY("London")
  }
};

------

p
minus
RELATION {
  TUPLE{
    PN("p1"), PNAME("Nut"),
    COLOR("Red"), WEIGHT(12), CITY("London")
  }
};
```

```
p
union
RELATION {
  TUPLE{
    PN("p7"), PNAME("Bult2"),
    COLOR("White"), WEIGHT(15), CITY("Kashan")
  },
  TUPLE{
    PN("p8"), PNAME("Clark"),
    COLOR("Red"), WEIGHT(20), CITY("London")
  }
};
```

```
RELATION {
  TUPLE{
    SN SN("s7"), SNAME SNAME("Clark"),
    STATUS STATUS(20), CITY CITY("London")
  },

  TUPLE{
    SN SN ("s8"), SNAME SNAME("John"),
    STATUS STATUS(25), CITY CITY("Shiraz")
  }
};

------

RELATION {
  TUPLE{
    SN("s4"), SNAME("Clark"),
    STATUS(20), CITY("London")
  },
  TUPLE{
    SN("s4"), SNAME("Clark"),
    STATUS(20), CITY("London")
  }
};
```

```
RELATION {
  TUPLE{SN SN("s4"), SNAME SNAME("Clark"), STATUS STATUS(20), CITY CITY("London")},
  TUPLE{SN SN ("s5"), SNAME SNAME("Adam"), STATUS STATUS(40), CITY CITY("London")}
  }
union
RELATION {
    TUPLE{SN SN("s6"), SNAME SNAME("Clark"), STATUS STATUS(20), CITY CITY("London")},
    TUPLE{SN SN ("s7"), SNAME SNAME("Adam"), STATUS STATUS(40), CITY CITY("London")
    }
};
```

Assignment

```
a := s
    union
      RELATION{
        TUPLE{SN("s16"), SNAME("Clark"), STATUS(20), CITY("London")},
        TUPLE{SN("s17"), SNAME("Adam"), STATUS(40), CITY("London")}
      };
```

SN	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens
s16	Clark	20	London
s17	Adam	40	London

Projection

```
s{city};
```

CITY
London
Paris
Athens

```
p{city} minus s{city};
```

CITY
Oslo

```
a minus s ;
```

SN	SNAME	STATUS	CITY
s16	Clark	20	London
s17	Adam	40	London

```
p{city, pname};
```

CITY	PNAME
London	Nut
Paris	Bolt
Oslo	Screw
london	Screw
Paris	Cam
London	Cog

شهرهایی را بیابید که هم عرضه کننده و هم قطعه در آنها هست.

```
s{city} minus (s{city} minus p{city});
```

```
p{city} minus (p{city} minus s{city});
```

```
s{city} intersect p{city};
```

CITY
London
Paris

شهرهای عرضه کنندگانی را بیابید که در آن شهرها قطعه‌ای نیست.

```
s{city} minus p{city};
```

CITY
Athens

شهرهایی را بیابید که یا عرضه کننده یا قطعه یا هر دو در آنهاست.

```
s{city} union p{city};
```

شهرهایی را بیابید که یا عرضه کننده یا قطعه در آنها باشد ولی هم قطعه و هم عرضه کننده در آن شهرها نباشد

```
(s{city} union p{city}) minus (s{city} intersect p{city});
```

```
(s{city} minus p{city}) union (p{city} minus s{city});
```

CITY
Athens
Oslo

Condition

```
p where city = "Paris";
```

PN	PNAME	COLOR	WEIGHT	CITY
P2	Bolt	Green	17	Paris
P5	Cam	Blue	12	Paris

نام عرضه کنندگان با وضعیت بیشتر از ۴۰ را بیابید.

```
s where status > 40;
```

```
(s where status > 40){sname};
```

نام عرضه کنندگانی را بیابید که وضعیت آنها(status) بیشتر از ۲۵ است.

```
(s where status > 25){sname};
```

Where

SN	SNAME	STATUS	CITY
S3	Blake	30	Paris
S5	Adams	30	Athens

Project

SNAME
Blake
Adams

Get part names of P2

نام قطعهٔ با شمارهٔ P2 را بیابید.

```
(P where pn = "P2") {pname};
```

نام شهرهایی را بیابید که یا قطعه‌ای با وزن بیشتر از 13 در آنها هست یا عرضه‌کننده‌ای با وضعیت بیشتر از ۳۴ در آنها هست.

1. نام شهرهایی را بیابید که قطعه‌ای با وزن بیشتر از 13 در آنها هست
2. نام شهرهایی را بیابید که عرضه‌کننده‌ای با وضعیت بیشتر از ۳۴ در آنها هست.

```
(p where weight > 13){city};

(s where status > 34){city};
```

```
(p where weight > 13){city}
union
(s where status > 34){city};
```

CITY
Paris
Oslo
london

نام شهرهایی را بیابید که یا قطعه‌ای با وزن بیشتر از ۲۴ در آنها هست یا عرضه‌کننده‌ای با وضعیت بیشتر از ۲۱ در آنها هست.

1. نام شهرهایی را بیابید که قطعه‌ای با وزن بیشتر از ۲۴ در آنها هست
2. نام شهرهایی را بیابید که عرضه‌کننده‌ای با وضعیت بیشتر از ۲۱ در آنها هست.

```
(p where weight > 24){city}


(s where status > 21){city};
```

```
(p where weight > 24){city}
union
(s where status > 21){city};
```

CITY
Paris
Athens

زوج شمارهٔ عرضه‌کنندگان و شمارهٔ قطعاتی را بیابید که آن عرضه کنندگان آن قطعات را عرضه کرده باشند.

- ```
  sp{sn , pn};
  ```
  ```

Times

```
p{city} times s{status};

p{pname, weight} times s{sname} ;

p{pn} times s{sn} ;

p{city} times s{city} -- wrong

p{pn, pname, color}
times
s{sn, sname, status};
```

CITY	STATUS
London	20
London	10
London	30
Paris	20
Paris	10
Paris	30
Oslo	20
Oslo	10
Oslo	30

زوج شمارهٔ عرضه‌کنندگان و قطعاتی را بیابید که آن عرضه کنندگان آن قطعات را عرضه نکرده باشند.

PN	SN
P1	S3
P1	S4
P1	S5
P2	S5
P3	S2
P3	S3

.

PN	SN
P3	S4
P3	S5
P4	S2
P4	S3
P4	S5
P5	S2

.

PN	SN
P5	S3
P5	S5
P6	S2
P6	S3
P6	S4
P6	S5

.

```
(p{pn} times s{sn})
minus
sp{sn,pn};
```

Rename

تغيير نام

```
S rename SN as SN1;
```

SN1	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

قطعه‌های عرضه شده را بیابید

```
p times sp;
-- wrong
```

```
p
times
(sp rename pn as sppn)
-- need another step
```

```
(
  p
  times
  (sp rename pn as sppn)

) where pn = sppn;
```

```
(
  (
    p
    times
    (sp rename pn as sppn)
  ) where pn = sppn
) {pn, pname, weight, color, city};
```

COLOR	PNAME	PN	WEIGHT	CITY
Red	Nut	P1	12	London
Green	Bolt	P2	17	Paris
Blue	Screw	P3	17	Oslo
Red	Screw	P4	14	london
Blue	Cam	P5	12	Paris
Red	Cog	P6	19	London

نام قطعه‌های عرضه شده را بیابید

```
p
times
(sp rename pn as sppn);
```

```
(
  p
  times
  (sp rename pn as sppn)
 ) where pn = sppn;
```

```
(
  (
    p
    times
    (sp rename pn as sppn)
  ) where pn = sppn
) {pname};
```

PNAME
Nut
Bolt
Screw
Cam
Cog

زوج شهرهای عرضه‌کنندگان و شهرهای قطعات را بیابید.

```
s times  p;
```

```
s{city} times  p{city};
```

```
( s rename city as scity ){scity}
times
p{city};
```

scity	CITY
London	London
London	Paris
London	Oslo
Paris	London
Paris	Paris
Paris	Oslo
Athens	London
Athens	Paris
Athens	Oslo

زوج شهرهای عرضه‌کنندگان و شهرهای قطعات غیرهمشهری را بیابید.

```
( s rename city as scity ){scity}
times
p{city};
```

```
(
  ( s rename city as scity ){scity}
  times
  p{city}
) where scity <> city;
```

scity	city
London	Paris
London	Oslo
Paris	London
Paris	Oslo
Athens	London
Athens	Paris
Athens	Oslo

زوج نام عرضه‌کنندگان و نام قطعاتی را بیابید که آن عرضه کننده آن قطعه را عرضه نکرده باشد (I)

```
(p{pn} times s{sn})
minus
sp{sn,pn};
```

```
( (p{pn} times s{sn})  minus  sp{sn,pn} )
times
(s{sn, sname} rename sn as ssn)

;
```

```
( (p{pn} times s{sn})  minus  sp{sn,pn} )
times
(s{sn, sname} rename sn as ssn)
times
(p{pn, pname} rename pn as ppn)
;
```

```
(
  (
    ( (p{pn} times s{sn})  minus  sp{sn,pn}
    times
    (s{sn, sname} rename sn as ssn)
  ) where sn = ssn
)
times
(p{pn, pname} rename pn as ppn)
;
```

ام عرضه‌کنندگان و نام قطعاتی را بیابید که آن عرضه کننده آن قطعه را عرضه نکرده باشد (II)

```
(
  (
    (
      ( (p{pn} times s{sn})  minus  sp{sn,pn} )
      times
      (s{sn, sname} rename sn as ssn)
    ) where sn = ssn
  )
  times
    (p{pn, pname} rename pn as ppn)
) where pn = ppn
;
```

```
(
  (
    ( (p{pn} times s{sn})  minus  sp{sn
    times
    (s{sn, sname} rename sn as ssn)
    times
    (p{pn, pname} rename pn as ppn)
  ) where pn = ppn and sn = ssn
)
{sname, pname}
;
```

زوج نام عرضه‌کنندگان و نام قطعاتی را بیابید که آن عرضه کننده آن قطعه را عرضه نکرده باشد(III)

```
(
  (
    ( (p{pn} times s{sn})  minus  sp{sn,pn} )
    times
    (s{sn, sname} rename sn as ssn)
    times
    (p{pn, pname} rename pn as ppn)
  ) where pn = ppn and sn = ssn
)
{sname, pname}
;
```

```
A := p{pn, pname} times s{sn, sname};
B := (sp{sn, pn} times (s{sn, sname} rename sn as ssn1) where sn = ssn1;
C := (B times (p{pn, pname} rename pn as pn1) where pn = pn1;
A minus (C{sn, sname, pn, pname})
```

زوج نام عرضه‌کنندگان و نام قطعاتی را بیابید که آن عرضه کننده آن قطعه را عرضه نکرده باشد (IV)

```
A := p{pn, pname} times s{sn, sname};
B := (
        sp{sn, pn}
        times
        (
          s{sn, sname} rename sn as ssn1
        ) where sn = ssn1
      )
    ;
C := (
        B
        times
        (
          p{pn, pname} rename pn as pn1
        ) where pn = pn1
      );
A minus (C{sn, sname, pn, pname})
```

نام عرضه کنندگانی را بیابید که قطعه‌ای عرضه کرده باشند.

```
s times sp
```

```
(s rename sn as sn1) times sp
```

```
((s rename sn as sn1) times sp) where sn1=sn;
```

```
(((s rename sn as sn1) times sp) where sn1=sn){sname};
```

sname
Smith
Jones
Blake
Clark

نام عرضه کنندگانی را بیابید که عرضه‌ای(qty) بیشتر از ۳۰۰ داشته باشند.

```
s
times
sp
```

```
(s rename sn as sn1)
times
sp
```

```
(s rename sn as sn1)
times
(sp where qty > 300)
```

```
(
  (s rename sn as sn1)
  times
  (sp where qty > 300)
) where sn1 = sn
```

```
(
  (
    (s rename sn as sn1)
    times
    (sp where qty > 300)
  ) where sn1 = sn
) {sname};
```

```
(
  (
    (s rename sn as sn1)
    times
    sp
  ) where sn1 = sn and qty > 300
) {sname};
```

نام عرضه کنندگانی را بیابید که قطعهٔ P4 را عرضه کرده باشد

```
(
  (
    (S rename sn as sn1)
    times
    (sp where pn = "P4")
  ) where sn1=sn
){sname};
```

sname
Smith
Clark

شهر عرضه کنندگانی را بیابید که قطعهٔ قرمزی را عرضه کرده باشند(I).

شهر عرضه‌کنندگانی را بیابید که دست کم یک عرضه‌کننده در آن شهر دست‌کم یک قطعهٔ قرمز را عرضه کرده باشد

```
(s rename sn as sn1)
times
sp
```

```
(
  (s rename sn as sn1)
  times
  sp
) where sn1 = sn
```

```
(
  (
    (s rename sn as sn1)
    times
    sp
  ) where sn1 = sn
)
times
(p rename pn as pn1)
```

```
(
  (
    (s rename sn as sn1)
    times
    sp
  ) where sn1 = sn
)
times
( (p rename pn as pn1) where color = "Red")
```

شهر عرضه کنندگانی را بیابید که قطعهٔ قرمزی را عرضه کرده باشند(II).

```
(
  (
    (s rename sn as sn1)
    times
    sp
  ) where sn1 = sn
)
times
(
  (
    (p rename pn as pn1)
    where color = "Red"
  ){pn1}
)
```

```
(
  (
    (
      (
        (s rename sn as sn1)
        times
        sp
      ) where sn1 = sn
    )
    times
    (
      (
        (p rename pn as pn1)
        where color = "Red"
      ){pn1}
    ) where pn1 = pn
  )
){city}
;
```

شهر عرضه کنندگانی را بیابید که قطعهٔ قرمزی را عرضه کرده باشند(III).

```
(
  (
    (
      (
        (
          (s rename sn as sn1)
          times
          sp
        ) where sn1=sn
      ){pn, city}
    )
    times
    (
      (
        (
          (p where color = "Red")
        ){pn}
    )
      ) rename pn as pn1
  ) where pn1=pn
){city};
```

CITY
London
Paris

شهر عرضه کنندگانی را بیابید که قطعهٔ قرمزی با وزن بیشتر از ۱۳ را عرضه کرده باشند.

```
(
  (
    (
      (
        (
          (s rename sn as sn1)
          times
          sp
        ) where sn=sn1
      ){pn, city}
    )
    times
    (
      (
        (
          (p where color = "Red")
          where weight > 13
        ){pn}
      ) rename pn as pn1
    )
  ) where pn1=pn
){city};
```

```
(
  (
    (
      (
        (s rename sn as sn1)
        times
        (sp rename pn as pn1)
      ) where sn1=sn
    ){pn1, city}
    times
    (
      (
        (p where color = "Red")
        where weight > 13
      ){pn}
    )
  ) where pn1=pn
){city};
```

شهر عرضه کنندگانی را بیابید که در شهر آنها قطعه‌ای نیز وجود داشته باشد.

```
s{city} intersect p{city};
```

نام عرضه کنندگانی را بیابید که در شهر آنها قطعه‌ای وجود ندارد.

```
s{city} minus p{city};
```

```
(s{city} minus p{city})
times
(s rename city as scity)
```

```
(
  (s{city} minus p{city})
  times
  (s rename city as scity)
) where city = scity
```

```
1  (
2   (
3      (s{city} minus p{city})
4      times
5      (s rename city as scity)
6    ) where city = scity
7  ){sname};
```

SNAME
Adams

نام قطعه‌های عرضه شده را بیابید.

```
(
  (
    p{pn, pname}
    times
    (sp rename pn as pn1)
  ) where pn1 = pn
) {pname};
```

PNAME
Nut
Bolt
Screw
Cam
Cog

شهر قطعه‌های عرضه شده با وزن بیشتر از ۱۵ را بیابید.

```
(
  (
    (p where weight > 15)
    times
    (sp rename pn as pn1)
  ) where pn1 = pn
){city};
```

CITY
Paris
Oslo
London

نام عرضه کنندگانی را بیابید که قطعه‌ای از شهر پاریس عرضه کرده باشند.

```
(
  (
    (
      (
        (
          s{sn, sname}
          times
          (sp rename sn as sn1)
        ) where sn1 = sn
      )
      times
      (p rename pn as pn1)
    ) where pn1 = pn
  ) where city = "Paris"
){sname} ;
```

SNAME
Smith
Jones
Blake
Clark

نام عرضه‌کنندگانی را بیابید که هیچ قطعه‌ای عرضه نکرده‌اند.

```
s{sn}  minus  sp{sn} ;
```

```
( s{sn}  minus  sp{sn} )
times s ; -- wrong
```

```
( s{sn}  minus  sp{sn} )
times
(s rename sn as ssn) ;
```

```
(
  ( s{sn}  minus  sp{sn} )
  times
  (s rename sn as ssn)
) where sn = ssn;
```

```
(
  (
    (s{sn}  minus  sp{sn})
    times
    (s rename sn as ssn)
  ) where sn = ssn
){sname}
;
```

```
s{sname} minus
(
  (
    s times
    (sp rename sn as sn1)
  ) where sn = sn1
){sname}
```

پیوند Join

```
( p times (sp rename pn as sppn) ) where sppn = pn
```

```
(
  p
  times
  (sp rename pn as sppn)
) where sppn = pn
```

```
p join sp
```

- Join does not have order
- It will apply equals for each shared attributes
- It will apply times for two relations without shared attribute

نام قطعه‌های عرضه شده را بیابید.

```
(
  (
    p
    times
    (sp rename pn as sppn)
  ) where sppn = pn
){pname};
```

```
(p join sp){pname}
```

نام قطعه‌های قرمز عرضه شده را بیابید.

```
(
  (
    (p where color="Red")
    times
    (sp rename pn as sppn)
  ) where sppn = pn
){pname};
```

```
(
  (p where color = "Red")
  join
  sp
){pname}
```

شهر قطعه‌های عرضه شده با وزن بیشتر از ۱۵ را بیابید.

```
(
  (p where weight > 15)
  join
  sp
){city};
```

CITY
Paris
Oslo
London

نام عرضه کنندگانی را بیابید که در شهر آنها قطعه‌ای وجود ندارد.

```
s{city} minus p{city};
```

```
(s{city} minus p{city})
join
s
```

```
(
  (s{city} minus p{city})
  join
  s
){sname};
```

SNAME
Adams

```
s{sname}
minus
((s join p){sname});
```

نام عرضه کنندگانی را بیابید که قطعه‌ای را از شهر پاریس عرضه کرده باشند.

```
( (s{sn, sname} join sp) join p) where city = "Paris"){sname
```

نام عرضه کنندگانی را بیابید که دست‌کم یکی از قطعه‌هایی که عرضه کرده‌اند در شهر پاریس باشد.

```
(
  (
    (s{sn, sname} join sp)
    join
    p
  ) where city = "Paris"
){sname} ;
```

SNAME
Smith
Jones
Blake
Clark

```
(                    --- راه حل نادرست
  (
   (s join sp)
   join p
  ) where city = "Paris"
){sname} ;
```

```
  (   -- راه حل درست
  ( (s join sp){pn, sname}
   join p
  ) where city = "Paris"
){sname} ;
```

```
  (     -- راه حل نادرست
   (p{city} where city='Paris') join s
){sname} ;
```

نام عرضه کنندگانی را بیابید که قطعه‌ای را در شهر خودشان عرضه کرده باشند.

SNAME
Smith
Jones
Blake
Clark

```
(
  (s join sp) join p
){sname} ;
```

```
--        نا درست۱
(
  (s join sp)
  join
  (p{city})
){sname} ;
```

```
--        نا درست۲
(
  (s join sp)
  join
  p{city}
){sname} ;
```

```
--        نا درست۳
(s join sp join p{city}){sname} ;
```

```
--        نا درست۴

((s join sp{sn})join p){sname};
```

```
--        نا درست۵
((sp join s){sn, sname} join p){sname};
```

```
--        نا درست۶
((sp join s){sn, sname, city) join p){sname};
```

```
--        نا درست۷
(((s join sp ){city}) join p){sname};
```

نام قطعاتی را بیابید که وزن‌شان بیشتر از ۱۰ است و عرضه‌کننده‌ای در شهر پاریس با وضعیت بیشتر از ۲۰ آنها را عرضه کرده باشد.

```
( p where weight > 10 ){pn, pname}
```

```
(s where status > 20) where city = "Paris"
```

```
( p where weight > 10 ){pn, pname}
join
sp
```

```
(
  (
    ( p where weight > 10 ){pn, pname}
    join
    sp
  )
  join
  (
    (s where status > 20) where city = "Paris"
  )
){pname} ;
```

PNAME
Bolt

Keys

Super Keys ابر کلیدها

- P(pn, pname, color, weight, city)
  - {pn}, {pn, pname}, {pn, color}, {pn, pname, weight}
  - {pn, city, color} {pn, pname, color, weight, city}
  - نادرست‌ها
    - {city}
    - {city, weight}
    - {pname, city, weight}
    - {pname, color}
  - هیچ دو قطعه‌ای در یک شهر نام یکسانی ندارند.
    - {pn}, {pname, city}, {pn, city}
    - {pname, city, weight}
  - اگر در هر شهر فقط یک قطعه بتواند باشد.
    - {pn}, {city}, {pn, city} , {city, pname}
    - {city, pname, color}

کلیدهای نامزد Candidate Keys

- S(sn, sname, status, city)
  - {sn}
  - هیچ دو عرضه‌کننده‌ای در یک شهر نام یکسانی ندارند.
    - {sn}
    - {sname, city}
  - اگر در هر شهر فقط یک عرضه کننده بتواند باشد.
    - {sn}
    - {city}
- SP(sn, pn, qty)
  - {sn, pn}

Rules for Identification

1. Uniqueness Property
2. Irreducibility (Minimality)
3. Functional Dependency
4. Existence

Primary key(PK)

- SP(sn, pn, qty)
- S(sn, sname, status, city)
- {sn}
- هیچ دو عرضه‌کننده‌ای در یک شهر نام یکسانی ندارند.
    - {sn}
    - {sname, city}
    - فقط یکی از این دو بالایی کلید اصلی گذاشته شود.
- اگر در هر شهر فقط یک عرضه کننده بتواند باشد.
    - {sn}
    - {city}
    - فقط یکی از این دو بالایی کلید اصلی گذاشته شود.

Guidelines for Optimal Selection

1. Derived from Candidate Keys
2. Attribute Stability
3. Minimality
4. High Frequency Usage
5. Non-Nullability

Surrogate Keys vs. Natural Keys

1. Avoid Arbitrary Fields
2. Criteria for Surrogate Keys
    - No natural candidate keys exist.
    - Existing candidate keys are excessively "wide"
    - Table structure, business rules, or attribute values are expected to change frequently in the future.
3. Surrogate Limitations
4. Last Resort Principle

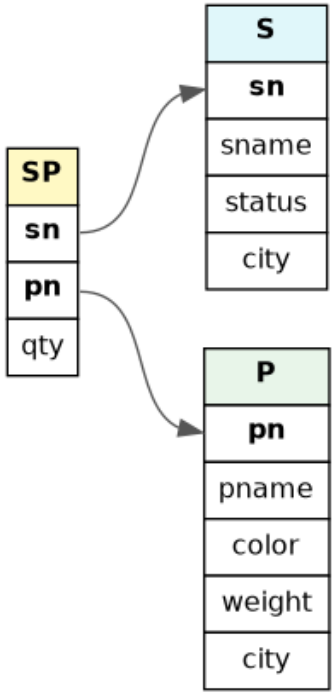
Common Mistakes to Avoid

1. Using Volatile Data
2. Using Large Strings
3. External Identifiers
4. Meaningful Key Updates

Foreign Key(FK)

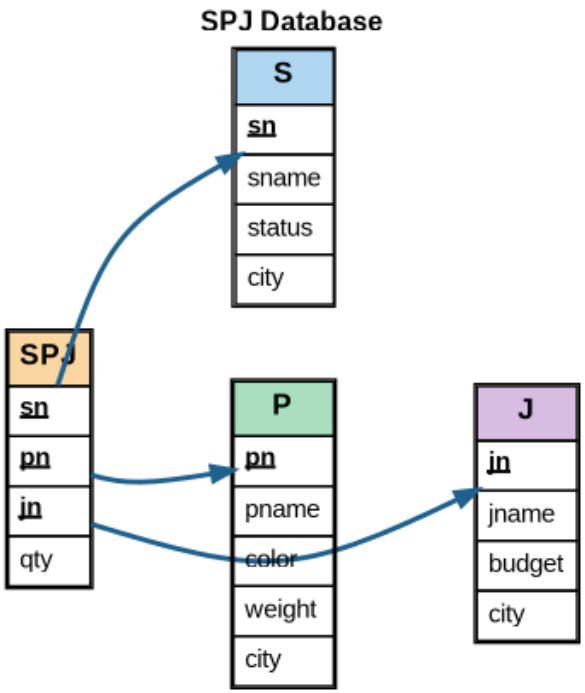
SP Database

- P(pn, pname, color, weight, city)
- S(sn, sname, status, city)
- SP(sn, pn, qty)



SPJ

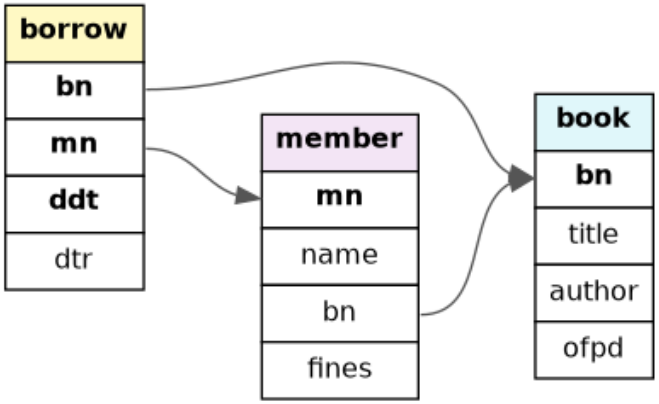
- S(sn,sname,status,city) ,
- P(pn,pname,color,weight,city) ,
- J(jn,jname,budget,city)
- SPJ(sn, pn, jn, qty)



Library

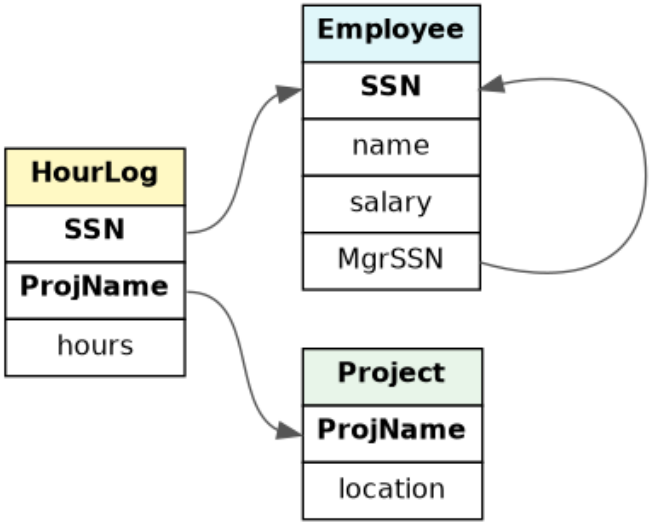
- book(bn, title, author, ofpd)
- member(mn, name, bn, fines)
- borrow(bn, mn, ddt, dtr)

1. "ofpd" overdue fine per day
2. "fines" Outstanding Balance or debt of a member
3. "ddt" due date
4. "dtr" date returned



Company Database Schema

- Employee, Project, HourLog
- Employee(SSN, name, salary, MgrSSN)
- Project(ProjName, location)
- HourLog(SSN, ProjName, hours)
- MgrSSN is foreign Key to SSN of its own table
- If the ProjName can be repeated?



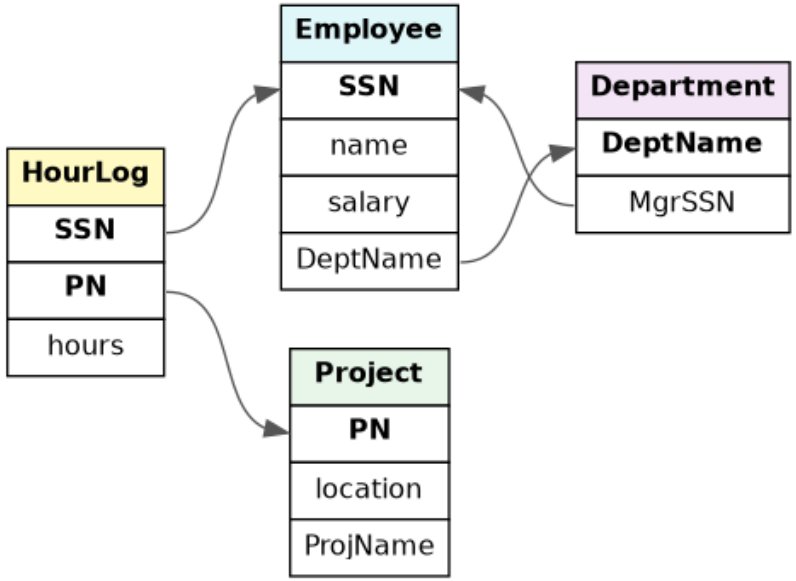
Company, Project Name duplication

- Employee(SSN, name, salary, MgrSSN)
- Project(ProjName, location)
- HourLog(SSN, ProjName, hours)

- Employee(SSN, name, salary, MgrSSN)
- Project(PN, ProjName, location)
- HourLog(SSN, PN, hours)
- If the company gets larger and needs departments?

Company with Departments

1. Employee(SSN, name, salary, DeptName)
2. Department(DeptName, MgrSSN)
3. Project(PN, location, ProjName)
4. HourLog(SSN, PN, hours)



References

- abs.gov.au
  https://www.abs.gov.au/websitedbs/a3121120.nsf/home/statistical+language+-+quantitative+and+qualitative+data
- questionpro
  https://www.questionpro.com/blog/quantitative-data/
- https://www.ntu.edu.sg/home/ehchua/programming/sql/Relational_Database_Design.html
- https://dev.to/lmolivera/everything-you-need-to-know-about-relational-databases-3ejl
- https://relational.fit.cvut.cz
- https://dzone.com/refcardz/from-relational-to-graph-a-developers-guide?chapter=1
- https://www.ibm.com/support/knowledgecenter/zh/SSWU4L/Data/imc_Data/Data_q_a_watson_assistant/Relational_Tables371.html
- https://vincentarelbundock.github.io/Rdatasets/datasets.html
- https://docs.oracle.com/cd/E15276_01/doc.20/e13677/dialogs_data_modeling.htm
- http://www.acc.umu.se/~cthulhu/postgreSQL/tutorial/sql608.htm
- https://fmhelp.filemaker.com/help/18/fmp/en/index.html#page/FMP_Help/planning-databases.html
- https://en.wikipedia.org/wiki/Relational_database
- https://web.csulb.edu/colleges/coe/cecs/dbdesign/dbdesign.php?page=manymany.php
- A video of C.J.Date

END